

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA Nº 09

NOMBRE COMPLETO: VELARDE VALENCIA JOSUE

Nº de Cuenta: 320342330

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 07

SEMESTRE 2027-1

FECHA DE ENTREGA LÍMITE: 28/10/2026

CALIFICACIÓN: _____

EJERCICIOS DE SESIÓN:

1. Actividades realizadas. Una descripción de los ejercicios y capturas de pantalla de bloques de código generados y de ejecución del programa

Para el primer ejercicio retomé la nave espacial que había agregado en la práctica anterior, modificando su movimiento para que recorriera automáticamente una pista de carreras. Para esto creé la función CrearPista(), en la que definí diferentes puntos de control para formar un circuito con rectas, curvas y un tramo diagonal, además de agregar una rampa de subida y otra de bajada mediante cambios en las coordenadas del eje Y.

Posteriormente utilicé estos puntos para generar los vértices e índices de la pista, creando una nueva malla mediante CreateMeshModel y agregándola al arreglo meshListModel para poder dibujarla en OpenGL.

```
void CrearPista()
{
    const GLfloat ANCHO_CIRCUITO = 10.0f;
    const GLfloat ALTURA_BAJA = 2.0f;
    const GLfloat ALTURA_ALTA = 14.0f;

    const glm::vec2 controlPista[][4] = {
        // Recta inferior con subida
        { {154.0f,306.0f}, {240.0f,306.0f}, {320.0f,306.0f}, {399.0f,306.0f} },
        // Curva inferior derecha
        { {399.0f,306.0f}, {428.0f,306.0f}, {445.0f,290.0f}, {445.0f,265.0f} },
        { {445.0f,265.0f}, {445.0f,234.0f}, {435.0f,211.0f}, {399.0f,211.0f} },
        // Recta central hacia la izquierda
        { {399.0f,211.0f}, {335.0f,211.0f}, {270.0f,211.0f}, {214.0f,211.0f} },
        // Curva interior de retorno
        { {214.0f,211.0f}, {192.0f,211.0f}, {181.0f,213.0f}, {181.0f,183.0f} },
        { {181.0f,183.0f}, {181.0f,163.0f}, {184.0f,150.0f}, {203.0f,146.0f} },
        // Recta diagonal interior
        { {203.0f,146.0f}, {270.0f,125.0f}, {338.0f,103.0f}, {402.0f,84.0f} },
        // Curva superior derecha
        { {402.0f,84.0f}, {420.0f,79.0f}, {430.0f,73.0f}, {429.0f,56.0f} },
        { {429.0f,56.0f}, {428.0f,42.0f}, {419.0f,36.0f}, {396.0f,36.0f} },
        // Recta superior
        { {396.0f,36.0f}, {303.0f,36.0f}, {212.0f,36.0f}, {120.0f,36.0f} },
        // Curva superior izquierda
        { {120.0f,36.0f}, {72.0f,36.0f}, {49.0f,63.0f}, {49.0f,110.0f} },
        // Lateral izquierdo con bajada
        { {49.0f,110.0f}, {49.0f,154.0f}, {49.0f,196.0f}, {49.0f,240.0f} },
        // Curva inferior izquierda
        { {49.0f,240.0f}, {49.0f,285.0f}, {75.0f,306.0f}, {122.0f,306.0f} },
        // Recta hasta la salida
        { {122.0f,306.0f}, {132.0f,306.0f}, {143.0f,306.0f}, {154.0f,306.0f} }
    };
    puntosCircuito.clear();
    const unsigned int MUESTRAS_POR_TRAMO = 64;
```

```
for (unsigned int tramo = 0; tramo < 14; tramo++)
{
    for (unsigned int i = 0; i < MUESTRAS_POR_TRAMO; i++)
    {
        GLfloat u = (GLfloat)i / MUESTRAS_POR_TRAMO;
        GLfloat v = 1.0f - u;
        glm::vec2 punto = v * v * v * controlPista[tramo][0]
            + 3.0f * v * v * u * controlPista[tramo][1]
            + 3.0f * v * u * u * controlPista[tramo][2]
            + u * u * u * controlPista[tramo][3];

        GLfloat altura = ALTURA_BAJA;
        if (tramo == 0) // Subida en la recta inferior
        {
            GLfloat t = glm::clamp((u - 0.15f) / 0.70f, 0.0f, 1.0f);
            t = t * t * (3.0f - 2.0f * t);
            altura = ALTURA_BAJA + (ALTURA_ALTA - ALTURA_BAJA) * t;
        }
        else if (tramo >= 1 && tramo <= 10) altura = ALTURA_ALTA;
        else if (tramo == 11) // Bajada en el lateral izquierdo
        {
            GLfloat t = glm::clamp((u - 0.15f) / 0.70f, 0.0f, 1.0f);
            t = t * t * (3.0f - 2.0f * t);
            altura = ALTURA_ALTA - (ALTURA_ALTA - ALTURA_BAJA) * t;
        }
        puntosCircuito.push_back(glm::vec3((punto.x - 240.0f) * 0.55f,
            altura, (punto.y - 170.0f) * 0.55f));
    }
}
distanciasCircuito.clear();
distanciasCircuito.push_back(0.0f);
```

```
    for (unsigned int i = 0; i < puntosCircuito.size(); i++)
    {
        distanciasCircuito.push_back(distanciasCircuito.back() + glm::length(
            puntosCircuito[(i + 1) % puntosCircuito.size()] - puntosCircuito[i]));
    }
    longitudCircuito = distanciasCircuito.back();

    std::vector<GLfloat> verticesCircuito;
    std::vector<unsigned int> indicesCircuito;
    for (unsigned int i = 0; i <= puntosCircuito.size(); i++)
    {
        unsigned int k = i % (unsigned int)puntosCircuito.size();
        glm::vec3 p = puntosCircuito[k];
        glm::vec3 tangente = glm::normalize(puntosCircuito[(k + 1) % puntosCircuito.size()] -
            puntosCircuito[(k + puntosCircuito.size() - 1) % puntosCircuito.size()]);
        glm::vec3 lateral = glm::normalize(glm::vec3(tangente.z, 0.0f, -tangente.x));
        glm::vec3 normal = glm::normalize(glm::cross(tangente, lateral));
        for (unsigned int j = 0; j < 2; j++)
        {
            glm::vec3 borde = p + lateral * (j == 0 ? ANCHO_CIRCUITO / 2.0f : -ANCHO_CIRCUITO / 2.0f);
            verticesCircuito.insert(verticesCircuito.end(), {
                borde.x, borde.y, borde.z,
                (GLfloat)j, (GLfloat)i / 8.0f,
                normal.x, normal.y, normal.z });
        }
        if (i < puntosCircuito.size())
        {
            unsigned int base = i * 2;
            indicesCircuito.insert(indicesCircuito.end(), { base, base + 1, base + 2,
                base + 1, base + 3, base + 2 });
        }
    }
    MeshModel* pista = new MeshModel();
    pista->CreateMeshModel(verticesCircuito.data(), indicesCircuito.data(),
        (unsigned int)verticesCircuito.size(), (unsigned int)indicesCircuito.size());
    indicePista = (unsigned int)meshListModel.size();
    meshListModel.push_back(pista);
}
```

Después implementé la función MoverNave(), utilizando los puntos del circuito y las distancias entre ellos para calcular la posición de la nave durante su recorrido. También calculé su orientación para que pudiera girar al llegar a las curvas y adaptarse a las rampas de subida y bajada. Finalmente utilicé matrizNave para aplicar estas transformaciones al modelo, haciendo que recorra la pista de manera continua y regrese al inicio al completar una vuelta.

```
glm::mat4 MoverNave(GLfloat pasoAnimacion)
{
    static GLfloat faseNave = 0.0f;
    //Recorrido de la nave sobre la linea central de la pista
    faseNave += 28.0f * pasoAnimacion;
    if (faseNave >= longitudCircuito) faseNave -= longitudCircuito;
    unsigned int indiceNave = 0;
    while (indiceNave + 2 < distanciasCircuito.size() &&
        faseNave >= distanciasCircuito[indiceNave + 1]) indiceNave++;
    GLfloat uNave = (faseNave - distanciasCircuito[indiceNave]) /
        (distanciasCircuito[indiceNave + 1] - distanciasCircuito[indiceNave]);
    glm::vec3 posicionNave = glm::mix(puntosCircuito[indiceNave],
        puntosCircuito[(indiceNave + 1) % puntosCircuito.size()], uNave);
    glm::vec3 avanceNave = glm::normalize(puntosCircuito[(indiceNave + 1) % puntosCircuito.size()] -
        puntosCircuito[(indiceNave + puntosCircuito.size() - 1) % puntosCircuito.size()]);
    glm::vec3 derechaNave = glm::normalize(glm::cross(glm::vec3(0.0f, 1.0f, 0.0f), avanceNave));
    glm::vec3 arribaNave = glm::normalize(glm::cross(avanceNave, derechaNave));
    glm::mat4 orientacionNave(1.0f);
    orientacionNave[0] = glm::vec4(derechaNave, 0.0f);
    orientacionNave[1] = glm::vec4(arribaNave, 0.0f);
    orientacionNave[2] = glm::vec4(avanceNave, 0.0f);
    glm::mat4 matrizNave = glm::mat4(1.0f);
    matrizNave = glm::translate(matrizNave, posicionNave + glm::vec3(0.0f, 1.5f, 0.0f));
    matrizNave = matrizNave * orientacionNave;
    matrizNave = glm::scale(matrizNave, glm::vec3(0.14f, 0.14f, 0.14f));

    return matrizNave;
}
```

Para el segundo ejercicio agregué un satélite formado por cuatro modelos diferentes: la base, la antena parabólica, el centro y los paneles solares. Cargué cada uno mediante la clase Model y utilicé una matriz principal llamada matrizSatelite, que contiene las transformaciones de posición y escala del satélite. A partir de esta matriz agregué las transformaciones correspondientes a sus componentes, conservando una jerarquía para que se mantengan unidos durante el movimiento.

Posteriormente implementé el desplazamiento automático del satélite mediante incrementos y decrementos en los ejes X y Z. Para esto utilicé condiciones que permiten cambiar de dirección al llegar a determinados límites, haciendo que el modelo siga una trayectoria rectangular de forma continua.

También agregué la animación de sus componentes, reemplazando los movimientos que anteriormente se realizaban mediante el teclado. Utilicé

anguloArticulaciones para controlar las rotaciones de la antena, el centro y los paneles, aumentando y disminuyendo su valor entre ciertos límites. Además, agregué una rotación continua a la antena parabólica sobre el eje Y utilizando glfwGetTime().

```
//Satelite: movimiento y matriz principal de toda la jerarquia
static glm::vec3 posicionSatelite(-16.0f, 20.0f, -16.0f);
static int tramoSatelite = 0;
static GLfloat anguloArticulaciones = 0.0f;
static GLfloat velocidadArticulaciones = 30.0f;
if (tramoSatelite == 0)
{
    posicionSatelite.x += 4.0f * pasoAnimacion;
    if (posicionSatelite.x >= 16.0f) { posicionSatelite.x = 16.0f; tramoSatelite = 1; }
}
else if (tramoSatelite == 1)
{
    posicionSatelite.z += 4.0f * pasoAnimacion;
    if (posicionSatelite.z >= 8.0f) { posicionSatelite.z = 8.0f; tramoSatelite = 2; }
}
else if (tramoSatelite == 2)
{
    posicionSatelite.x -= 4.0f * pasoAnimacion;
    if (posicionSatelite.x <= -16.0f) { posicionSatelite.x = -16.0f; tramoSatelite = 3; }
}
else
{
    posicionSatelite.z -= 4.0f * pasoAnimacion;
    if (posicionSatelite.z <= -16.0f) { posicionSatelite.z = -16.0f; tramoSatelite = 0; }
}
anguloArticulaciones += velocidadArticulaciones * pasoAnimacion;
if (anguloArticulaciones >= 30.0f) { anguloArticulaciones = 30.0f; velocidadArticulaciones = -30.0f; }
if (anguloArticulaciones <= -30.0f) { anguloArticulaciones = -30.0f; velocidadArticulaciones = 30.0f; }
glm::mat4 matrizSatelite = glm::mat4(1.0f);
matrizSatelite = glm::translate(matrizSatelite, posicionSatelite);
matrizSatelite = glm::scale(matrizSatelite, glm::vec3(0.40f, 0.40f, 0.40f));
```

```
//Base del satelite
color = glm::vec3(0.5f, 0.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glm::mat4 matrizBaseSatelite = glm::rotate(matrizSatelite, glm::radians(180.0f), glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(matrizBaseSatelite, glm::vec3(100.0f, 100.0f, 100.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
satelitebase.RenderModel();

//Antena
color = glm::vec3(0.3f, 0.3f, 0.3f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
model = glm::translate(matrizSatelite, glm::vec3(-7.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(40.0f * (GLfloat)glfwGetTime()), glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, glm::radians(anguloArticulaciones), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(100.0f, 100.0f, 100.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
parabolica.RenderModel();

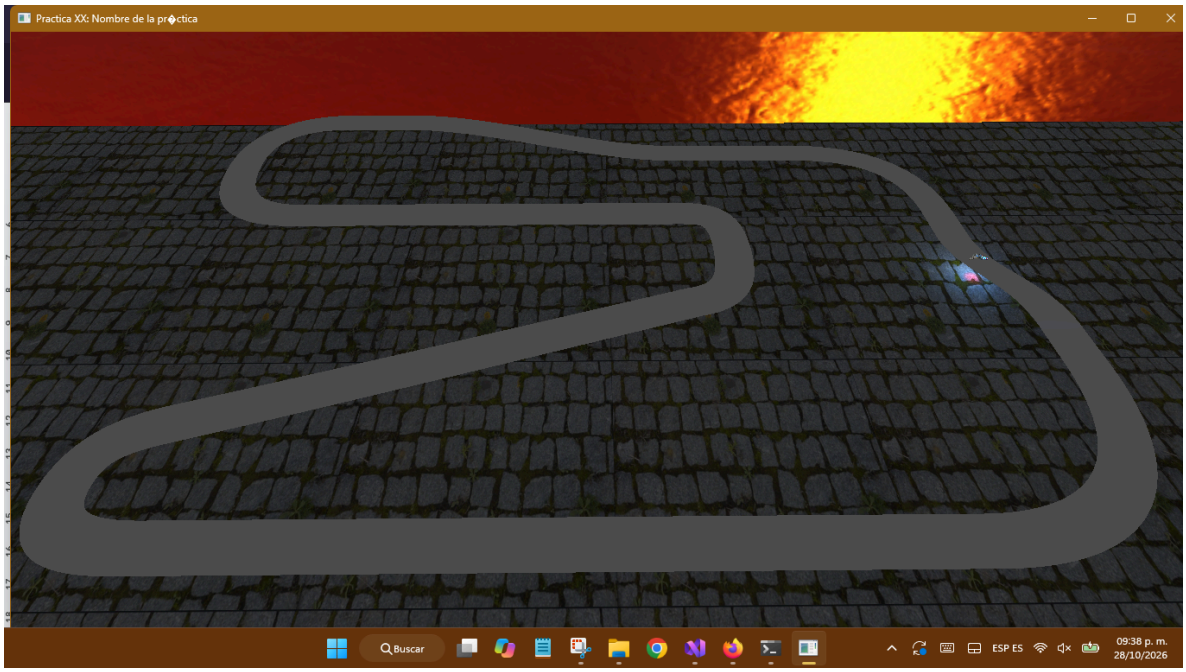
//Centro
color = glm::vec3(0.8f, 0.3f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glm::mat4 matrizCentroSatelite = glm::translate(matrizSatelite, glm::vec3(-2.0f, 0.0f, 0.0f));
matrizCentroSatelite = glm::rotate(matrizCentroSatelite, glm::radians(180.0f), glm::vec3(0.0f, 1.0f, 0.0f));
matrizCentroSatelite = glm::rotate(matrizCentroSatelite, glm::radians(anguloArticulaciones * (2.0f / 3.0f)), glm::vec3(-1.0f, 0.0f, 0.0f));
model = glm::scale(matrizCentroSatelite, glm::vec3(100.0f, 100.0f, 100.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
centro.RenderModel();
```

```
//Panel izquierdo
color = glm::vec3(0.0f, 1.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
model = glm::translate(matrizCentroSatelite, glm::vec3(0.0f, 0.0f, -6.5f));
model = glm::rotate(model, glm::radians(180.0f), glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, glm::radians(anguloArticulaciones * (4.0f / 3.0f)), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(100.0f, 100.0f, 100.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
panel.RenderModel();

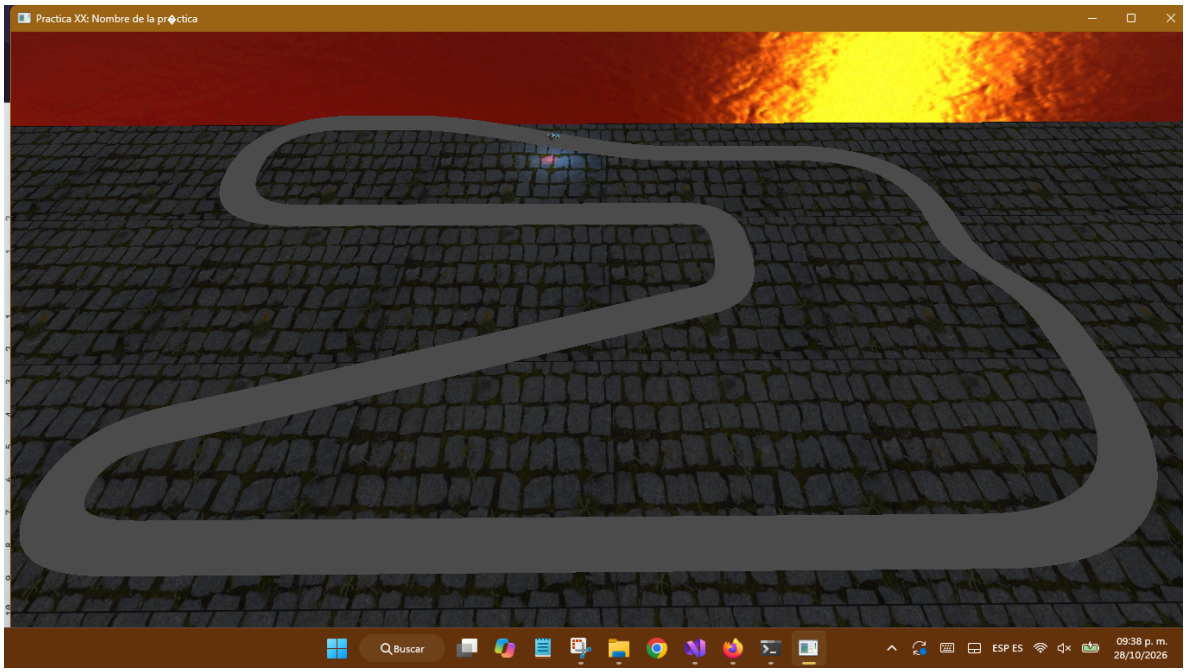
//Panel derecho
color = glm::vec3(1.0f, 1.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
model = glm::translate(matrizCentroSatelite, glm::vec3(0.0f, 0.0f, 6.5f));
model = glm::rotate(model, glm::radians(-anguloArticulaciones * (4.0f / 3.0f)), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(100.0f, 100.0f, 100.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
panel.RenderModel();

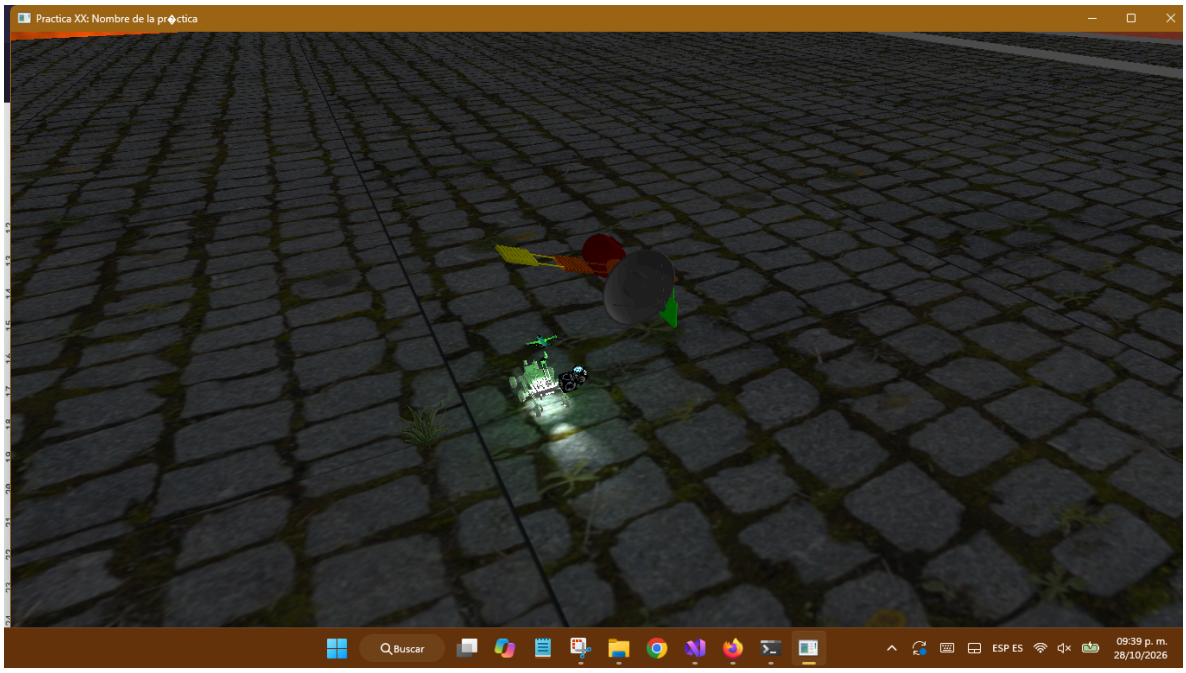
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
```

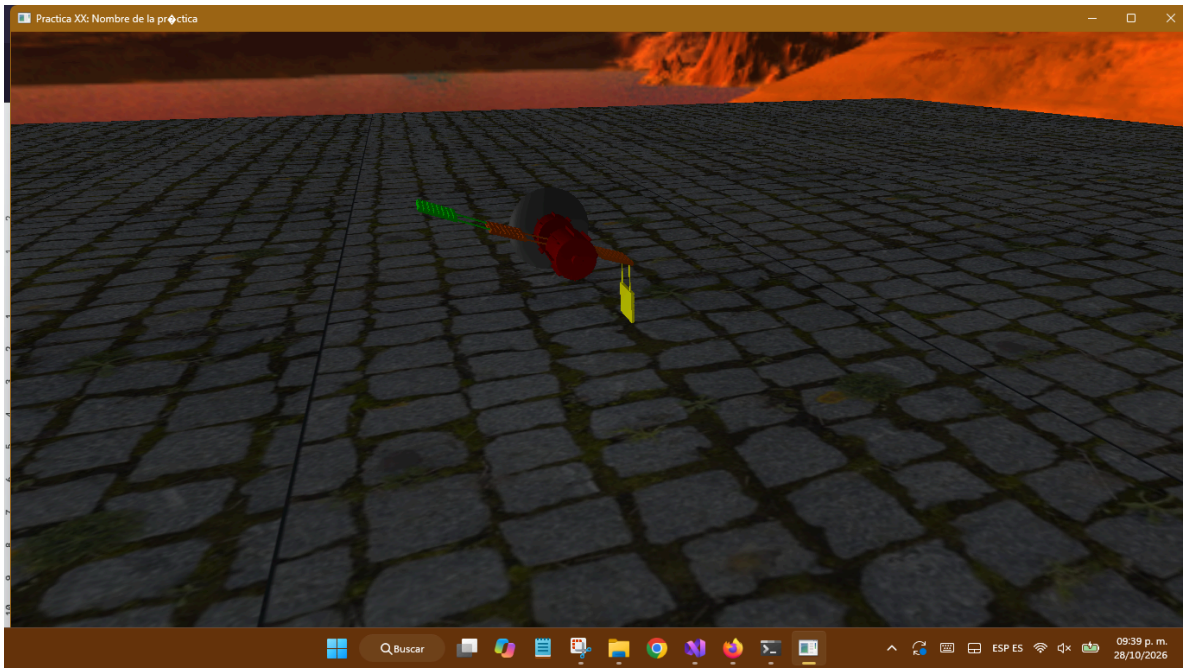
La ejecución final se muestra a continuación:




Practica XX: Nombre de la pr◆ctica
Buscar
ESP ES
09:38 p. m.
28/10/2026


Practica XX: Nombre de la pr◆ctica
Buscar
ESP ES
09:38 p. m.
28/10/2026


Practica XX: Nombre de la pr◆ctica
Buscar
ESP ES
09:39 p. m.
28/10/2026


Practica XX: Nombre de la pr◆ctica
Buscar
ESP ES
09:39 p. m.
28/10/2026



2. Problemas presentados.

Durante el desarrollo de la práctica fue necesario realizar algunos ajustes en las dimensiones de la pista y en la escala de los modelos para que la nave pudiera recorrer correctamente el circuito. También ajusté las transformaciones del satélite para conservar la jerarquía de sus componentes mientras realizaban sus movimientos automáticos.

3. Conclusión:

La realización de esta práctica me permitió comprender mejor cómo implementar animaciones automáticas y simples en OpenGL mediante el uso de variables, condiciones y matrices de transformación. La creación de la pista me ayudó a entender cómo relacionar el movimiento y la orientación de un modelo con una trayectoria definida, mientras que la animación del satélite me permitió reforzar el manejo de jerarquías para controlar varios componentes de manera conjunta. Con estos ejercicios pude aplicar diferentes transformaciones para generar movimientos continuos sin necesidad de utilizar el teclado.